Eye-Controlled Cursor

Phase 1 Report

team: pola raouf malak /ahmed tamer / eyad ashraf

Instructor: dr:salwa osama

1. Problem Definition & Objectives

Problem Statement:
Many individuals with physical disabilities face difficulties using traditional input devices such as mice and keyboards. This limits their ability to interact with computers efficiently and independently.

Objectives:
- Develop a system that allows users to control the computer cursor using eye movement.
- Improve accessibility for users with limited mobility.
- Provide a natural and intuitive interaction method between the user and the computer.

Target Users:
- Individuals with motor disabilities.
- Users seeking hands-free interaction with computers.

HCI Relevance:
The system enhances Human-Computer Interaction by providing an alternative input method, allowing users to interact with digital environments through eye movements, aligning with principles of usability and accessibility.

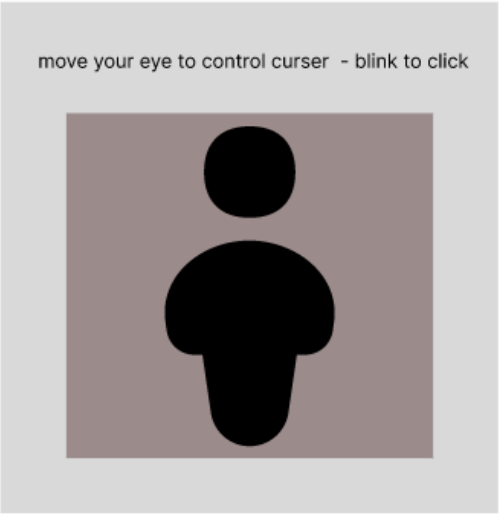
2. System Design / Architecture

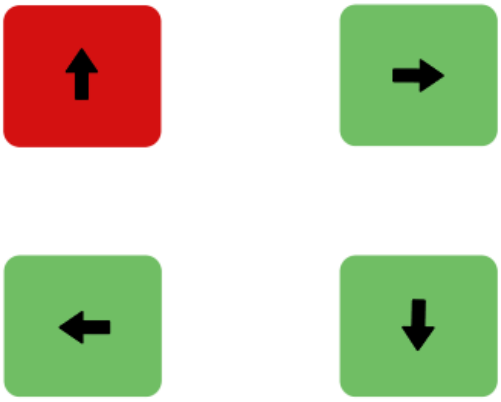
System Flow:
1. Capture video feed from the webcam.
2. Detect the face within the video frame.
3. Detect and track eye positions.
4. Analyze eye movements to determine cursor direction.
5. Move the cursor accordingly.

Block Diagram:
Camera -> Face Detection -> Eye Tracking -> Direction Analysis -> Cursor Movement

Inputs: Webcam video feed.
Outputs: Computer cursor movement on the screen.

3. User Interface Design (Prototype)





4. Tools & Technology Plan

Tool / Library	Purpose	Reason
Python	Main programming language	Easy to implement image processing
OpenCV	Face and eye detection	Provides robust tools for real-time video analysis
Mediapipe	Facial landmark detection	Simplifies eye tracking and provides high accuracy

PyAutoGUI	Control mouse cursor	Allows direct control of cursor based on computed direction
Kivy (optional)	GUI for the application	Lightweight and supports Python-based UI design

5. Team Responsibilities

pola raouf : problem definition & objectives - concept code

Ahmed tamer : System Design & Architecture - Prototype

eyad ashraf : tools&technology

6. Concept Code

```
import cv2


cap = cv2.VideoCapture(0)
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')

while True:
    ret, frame = cap.read()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    for (x, y, w, h) in faces:
        roi_gray = gray[y:y+h, x:x+w]
        eyes = eye_cascade.detectMultiScale(roi_gray)
        for (ex, ey, ew, eh) in eyes:
            cv2.rectangle(frame, (x+ex, y+ey), (x+ex+ew, y+ey+eh), (0,255,0), 2)

    cv2.imshow('Eye Detection', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```